Yolande Ekpe, aneekpe@kth.se
Sahand Khodabandehloo, sahandkh@kth.se

Computer Laboratory Report: Molecular Simulation

This computer laboratory is about simulating the behavior of 40 argon atoms in a 2-dimensions box of 4 nm length at a temperature of 300 K for 50 ps.

a) Algorithm to solve Newton's equations of motion

Expression of the position x(t) and velocity v(t) for the algorithm in the script

In the Python script, the position, $\vec{x}_i(t)$, and velocity, $\vec{v}_i(t)$, of each particle, i, are expressed as follows:

$$\begin{cases} x_i = x_i + v_{x_i} dt \\ y_i = y_i + v_{y_i} dt \end{cases} \quad (1)$$

$$\begin{cases} v_{x_i} = v_{x_i} + \frac{1}{2m} F_{x_i} dt, & v_{x_0} = 0 \\ v_{y_i} = v_{y_i} + \frac{1}{2m} F_{y_i} dt, & v_{y_0} = 0 \end{cases} \quad (2)$$

In vector form:

$$\vec{x}_i = \vec{x}_i + \vec{v}_i dt \quad (3)$$

$$\vec{v}_i = \vec{v}_i + \frac{1}{2m} \vec{F}_i dt \quad (4)$$

, where $\vec{x}_i = [x_i, y_i]^T$, $\vec{v}_i = [v_{x_i}, v_{y_i}]^T$, and $\vec{F}_i = [F_{x_i}, F_{y_i}]^T = m\frac{d\vec{v}_i}{dt}$ represents the force acting on the particle according to Newton II.

Since molecular dynamics simulations evolve in time, the code iteratively updates $\vec{x}_i$ and $\vec{v}_i$ at each step, which ensures efficient computation. This approach allows for real-time tracking of particle motion while minimizing memory usage, making it well-suited for large-scale simulations.

Yolande Ekpe, aneekpe@kth.se
Sahand Khodabandehloo, sahandkh@kth.se

Expression of the position x(t) and velocity v(t) for the Leap-frog algorithm

In the Leap-Frog algorithm, the position, $\vec{x}_i(t)$, and velocity, $\vec{v}_i(t)$, of each particle, i, are updated as follows:

$$\begin{cases} x_{i+1} = x_i + v_{x_{i+1/2}} dt \\ y_{i+1} = y_i + v_{y_{i+1/2}} dt \end{cases} \tag{5}$$

$$\begin{cases} v_{x_{i+1/2}} = v_{x_{i-1/2}} + a_{x_{i+1}} dt = v_{x_{i-1/2}} \dfrac{dt}{m} F_{x_{i+1}}, & v_{x_0} = 0 \\ v_{y_{i+1/2}} = v_{y_{i-1/2}} + a_{y_{i+1}} dt = v_{y_{i-1/2}} \dfrac{dt}{m} F_{y_{i+1}}, & v_{y_0} = 0 \end{cases} \tag{6}$$

In vector form:

$$\vec{x}_{i+1} = \vec{x}_i + \vec{v}_{i+1/2} dt \tag{7}$$

$$\vec{v}_{i+1/2} = \vec{v}_{i-1/2} + \frac{dt}{m} \vec{F}_{i+1} \tag{8}$$

, where $\vec{x}_i = [x_i, y_i]^T$, $\vec{v}_i = \left[v_{x_i}, v_{y_i}\right]^T$, $\vec{a}_i = \left[a_{x_i}, a_{y_i}\right]^T = \frac{d\vec{v}_i}{dt}$ is the acceleration vector, and $\vec{F}_i = \left[F_{x_i}, F_{y_i}\right]^T$, the force vector acting on the particle.

Drawbacks of the two algorithms

The algorithm in the script assumes that both velocity and force remain constant over each time step $[t, t + dt]$, i.e., acceleration does not change within this interval. Nevertheless, the local truncation error in each step is proportional to dt^2. However, since the total number of steps in a simulation is T/dt, where T is the total simulation time, the global error is proportional to dt. Consequently, Euler's method is unsuitable for long-time simulations, as larger time steps introduce significant accumulated inaccuracies and instability, while smaller time steps increase computational cost.

In contrast to Euler's first-order method, the Leap-Frog algorithm updates the positions at full-time steps $(t + dt)$ and the velocities at half-time steps $(t + dt/2)$. This staggered update reduces numerical errors and improves numerical stability by minimizing the number of approximated variables stored at once. While the method still depends on the time step for accuracy, its second-order nature results in a global error proportional to dt^2, making it more accurate and efficient for long-term simulations and energy conservation.

Yolande Ekpe, aneekpe@kth.se
Sahand Khodabandehloo, sahandkh@kth.se

b) Comparison of the simulation with different ε

Figures 1 and 2 illustrate how the Lennard-Jones parameter, ε, affects the energy distribution of the system. As ε increases, the potential energy also rises, indicating stronger binding interactions and a more stable system. According to the Lennard-Jones equation, only the potential energy, represented as $V(r)$ in the script, depends on ε, while the kinetic energy does not. Therefore, an increase in ε leads to a rise in the magnitude of potential energy, as illustrated in Figures 1 and 2. Since kinetic energy is not influenced by ε, any change in kinetic energy is negligible. This was confirmed in simulations at higher ε values, where kinetic energy nearly remained constant. Consequently, while the total energy remains conserved, as indicated by the straight green line, its magnitude decreases with the increase in ε.

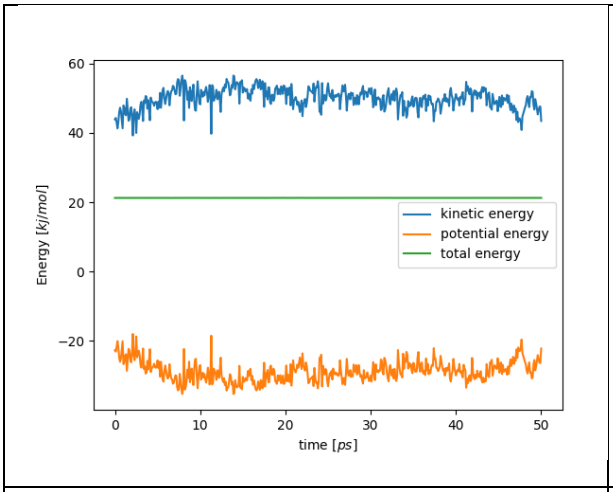


Figure 1: Energy for ε = 0.979 kJ/mol

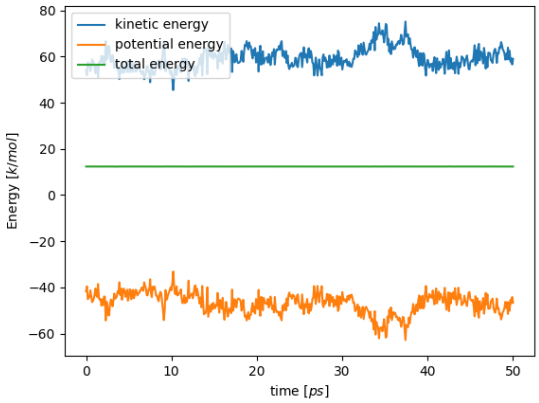


Figure 2: Energy for ε = 1.5 kJ/mol

Figure 3 shows that increasing ε results in a deeper potential well, indicating stronger intermolecular interactions between the argon atoms. This suggests a more stable bonding configuration, where the system exhibits greater structural integrity and is less susceptible to external disturbances (e.g., thermal fluctuations and force changes).

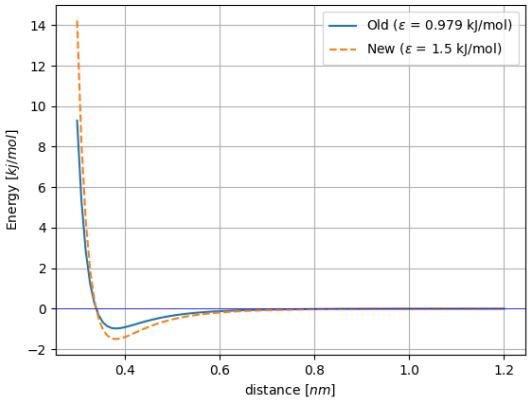


Figure 3: Potential energy as a function of distance

Yolande Ekpe, aneekpe@kth.se
Sahand Khodabandehloo, sahandkh@kth.se

c) Effect of time steps larger than 0.01 ps

Figures 4-7 illustrate the impact of time step dt selection on energy conservation and numerical stability by showing the evolution of the different energies over time for different values of dt.

For time steps 0.01 ps and 0.02 ps, kinetic and potential energies exhibit small fluctuations due to random particle collisions, causing an exchange of energy. Meanwhile, the total energy remains constant, indicating numerical stability and proximity to the stability threshold. However, since minor deviations of total energy appear for time step 0.03 ps, the stability threshold has been exceeded. Beyond this time step, instability becomes significantly emerging, rendering the simulation unreliable. Since numerical instability arises for 0.03 ps, the maximum time step should be chosen to be less than 0.03 ps.

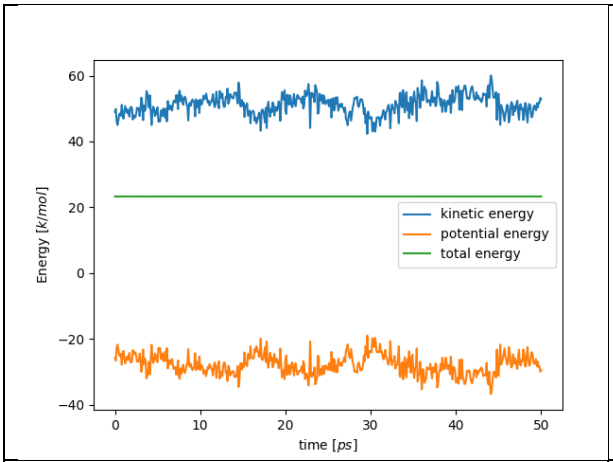


Figure 4: Energies with dt = 0.01 ps

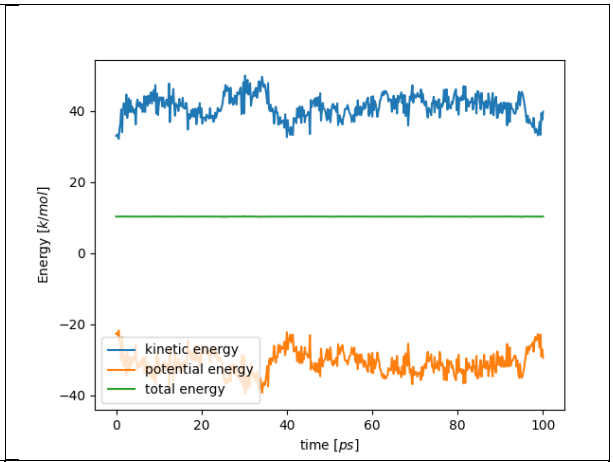


Figure 5: Energies with dt = 0.02 ps

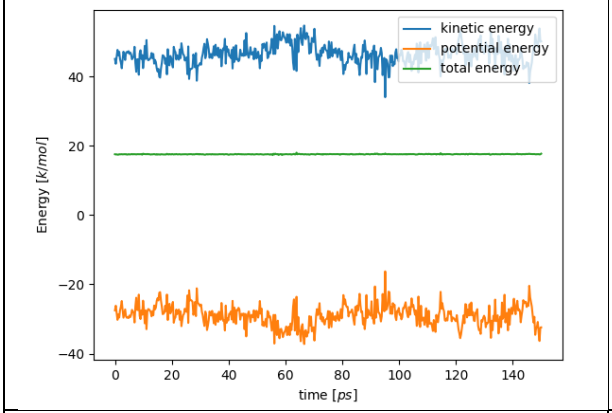


Figure 6: Energies with dt = 0.03 ps

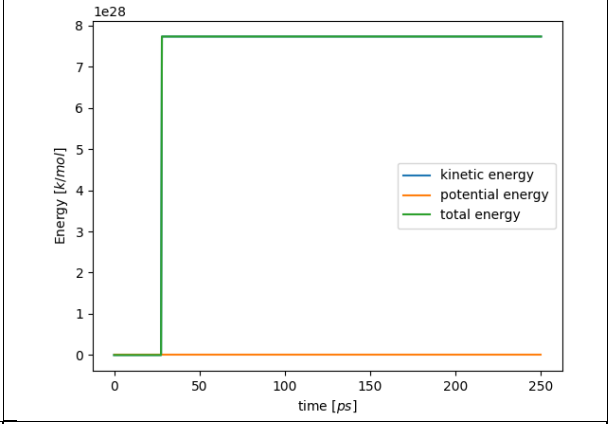


Figure 7: Energies with dt = 0.05 ps

Yolande Ekpe, aneekpe@kth.se
Sahand Khodabandehloo, sahandkh@kth.se

d) Effect of mass and temperature on energy

Simulation with different temperatures for the initial mass n = 40

Figures 8-11 highlight the effect of temperature, T, on energy conservation over time for various temperatures. The particle mass, m, and number of particles, n, remain constant to ensure that the total system mass does not change.

At a lower temperature (150 K), the magnitude of kinetic energy is smaller than that of potential energy, reflecting low molecular motion and strong intermolecular interactions. As temperature increases, kinetic energy rises while potential energy decreases (becoming less negative) due to weaker intermolecular forces. At a high temperature (900 K), kinetic energy dominates. However, across all figures, total energy remains conserved, demonstrating numerical stability.

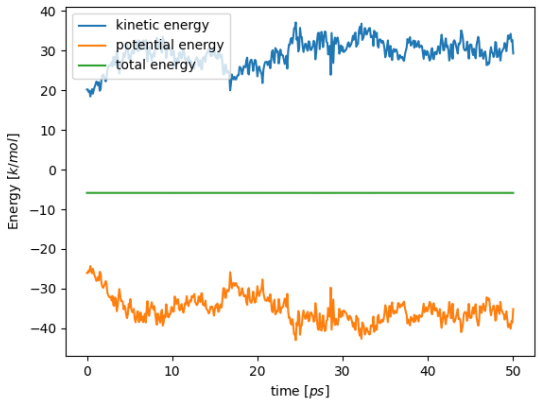


Figure 8: Energies for T = 150 K

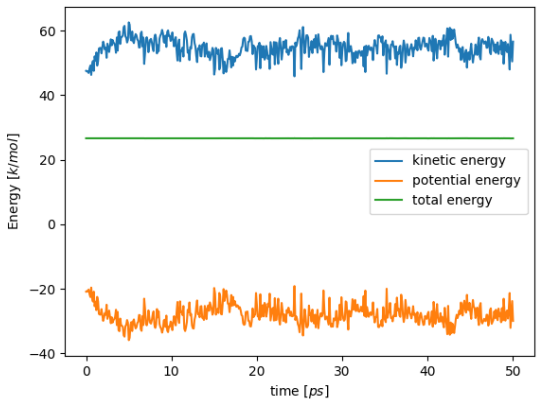


Figure 9: Energies for T = 300 K

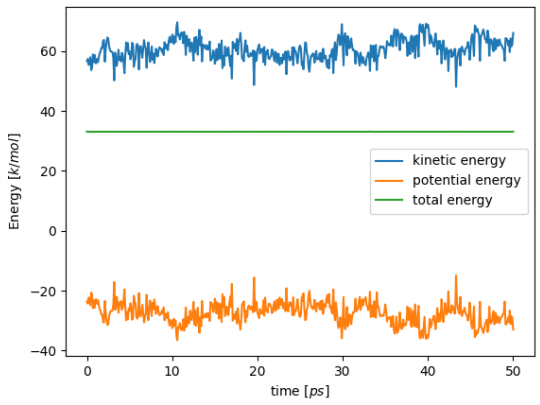


Figure 10: Energies for T = 450 K

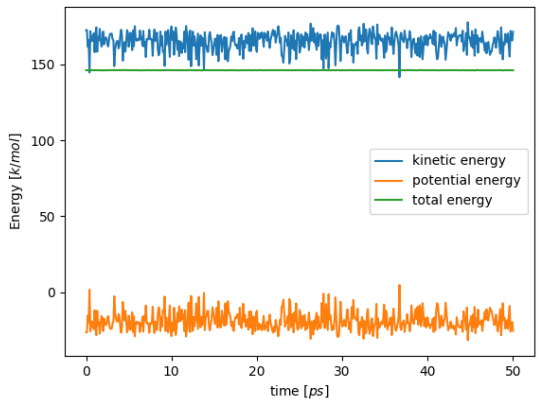


Figure 11: Energies for T = 900 K

Yolande Ekpe, aneekpe@kth.se
Sahand Khodabandehloo, sahandkh@kth.se

According to the equipartition theorem, the average kinetic energy of a particle with N_{DOF} degrees of freedom (DOF) is given by:

$$K = \frac{N_{DOF} k_B T}{2} \quad (9)$$

, where k_B is Boltzmann's constant and T is the absolute temperature. This indicates that, at constant mass, increasing temperature increases kinetic energy, as shown in Figures 8-11.

Simulation with different masses for temperature T = 300 K

Figures 12-15 underscore the influence of system mass, i.e., elements (with different masses), on energy conservation for a set of forty particles, $n = 40$, at a fixed temperature of 300 K.

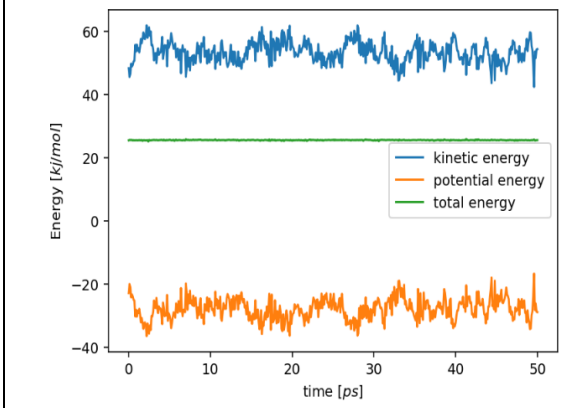


Figure 12: m = 4.002 u

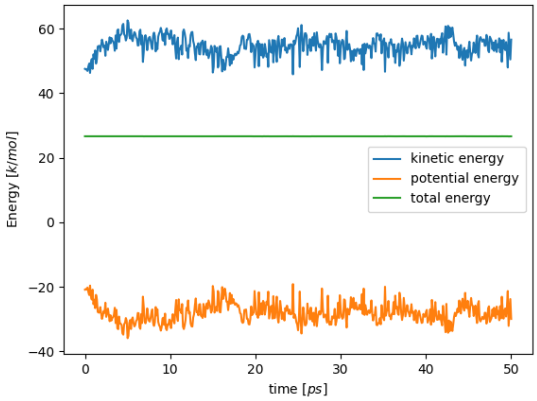


Figure 13: Argon, m = 39.948 u

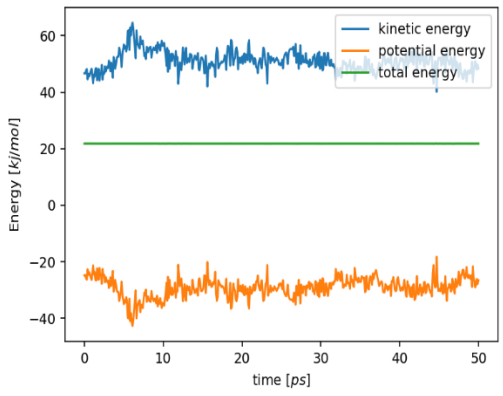


Figure 14: m = 83.798 u

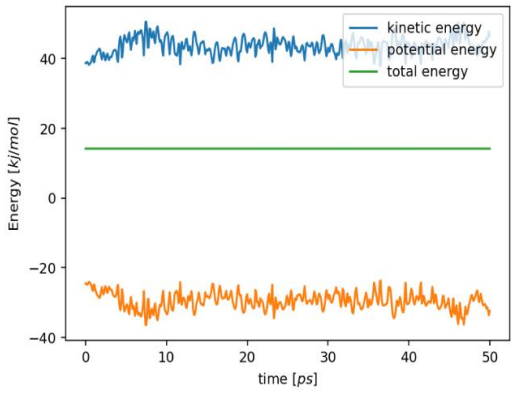


Figure 15: m = 131.293 u

For a particle with mass m and velocity v, kinetic energy K is given by:

$$K = \frac{1}{2} m v^2 \quad (10)$$

The total kinetic energy of the system is the sum of the kinetic energy of all particles:

$$K_{total} = \sum_{i=1}^{n} \frac{1}{2} m_i {v_i}^2 \quad (11)$$

According to (9), in a system at thermal equilibrium, i.e., constant temperature (300 K), the average kinetic energy per particle remains constant, regardless of changes in mass. However, the particle velocity does change, as can be seen by substituting (10) into (9):

$$v^2 = \frac{N_{DOF} k_B T}{m} \quad (12)$$

This demonstrates that, at a constant temperature, increasing mass must decrease velocity, i.e., molecular motion, to maintain the same kinetic energy. Figures 12-15 evidently illustrate this trend, as kinetic energy remains unchanged. Additionally, potential energy remains unaffected, implying not only that total energy is conserved but also that mass variations do not alter energy distribution. However, the fluctuations observed in the figures are likely to be due to numerical errors inherent in the simulation or thermal fluctuations arising from continuous energy exchange between particles during collisions.

Ultimately, as the time step determines how frequently positions and velocities of the atoms are updated, fewer atoms and higher temperatures require smaller time steps to track motion accurately. Consequently, such simulations require more computational power. Simulations use techniques like bond constraints to increase time steps and improve efficiency.

Yolande Ekpe, aneekpe@kth.se
Sahand Khodabandehloo, sahandkh@kth.se

e) Simulation method choice for the conformation of a small protein

The choice between Monte Carlo (MC) and Molecular Dynamic (MD)—stochastic sampling vs. determinism evolution—depends on the trade-off between computational efficiency and accuracy. Although MC excels in equilibrium sampling, provided sufficient simulation time, it lacks the ability to capture time-dependent properties necessary required to track system evolution. In conclusion, since protein conformational rearrangement is a time-dependent process, MD is the preferred method, as it accurately determines molecular motion using Newton's equations. Additionally, while a coarse-grained model improves efficiency for large systems, an atomistic model is more suitable for capturing detailed interactions, making it ideal for studying protein dynamics.